# STA 314: Statistical Methods for Machine Learning I

## Lecture - Bagging, random forest and boosting

Xin Bing

Department of Statistical Sciences
University of Toronto

# Review on simple decision trees

- Decision trees
  - Regression tree
  - Classification tree
- Self-explanatory
- Recursive binary splitting
- Could be deeply grown trees, with large variance
- Pruning

# Bagging

- **Bootstrap aggregation**, or **bagging**, is a general-purpose procedure for reducing the variance of a statistical learning method

Two heads are better than one!

**三个臭皮匠，顶个诸葛亮**

- We introduce it here because it is particularly useful and frequently used in the context of decision trees.

# Why bagging? The idea of bagging.

- Given a set of $N$ independent estimates $\hat{\beta}^{(1)}, \ldots, \hat{\beta}^{(N)}$ of $\beta$, suppose

$$\left|\mathbb{E}[\hat{\beta}^{(i)}] - \beta\right| \leq b, \qquad \mathrm{Var}(\hat{\beta}^{(i)}) = \sigma^2, \quad \forall i = 1, \ldots, N.$$

Then the averaged estimate

$$\bar{\beta} := \frac{1}{N}\sum_{i=1}^{N} \hat{\beta}^{(i)}$$

$\mathbb{E}(\bar{\beta} - \beta)^2$

$= (\mathbb{E}\bar{\beta} - \beta)^2$

satsifying

$$\left|\mathbb{E}[\bar{\beta}] - \beta\right| \leq b, \qquad \mathrm{Var}(\bar{\beta}) = \frac{\sigma^2}{N}.$$

$+ \mathrm{Var}(\bar{\beta})$

- In other words, averaging a set of **independent** random variables reduces the variance, meanwhile does not incur extra bias.

# Bagging

- To apply bagging to regression trees, if we had $B$ independent training sets, then we simply construct $B$ regression trees using each of the training set, and average the resulting predictions in the end.

- Since we only have one training set, we **bootstrap** the same amount of data $B$ times from the training set **with replacement**

- We train a decision tree by using the $b$th boostrap data and get the prediction $\hat{f}^{*b}(x)$ at a point $x$. Bagging averages all the predictions

$$\hat{f}_{bag}(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^{*b}(x).$$

# Bagging Trees

- The fitted decision trees on boostrap data sets are grown deep, and need not be pruned. Hence each individual tree has **high variance**, but **low bias**. Averaging these trees reduces the variance.

- Bagging can be also applied to classification trees. For a given test observation, we can record the class predicted by each of the $B$ trees, and take a majority vote.

# Out-of-Bag Error Estimation

- There is a simple way to estimate the test error of a bagged model, without the need to perform cross-validation or the validation set approach.

w.r.

- For each bootstrap data set,
  - It only contains, on average, around 2/3 of the original observations.
  - The remaining 1/3 is not used to fit the decision tree, and is referred to as the **out-of-bag (OOB) observations**.
  - We predict the OOB observations by using the fitted decision tree.

# Out-of-Bag Error Estimation

$$\frac{1}{n}\sum_{i=1}^{n}\left(y_i - \frac{1}{|B_i|}\sum_{j\in B_i}\hat{f}_j(x_i)\right)^2$$

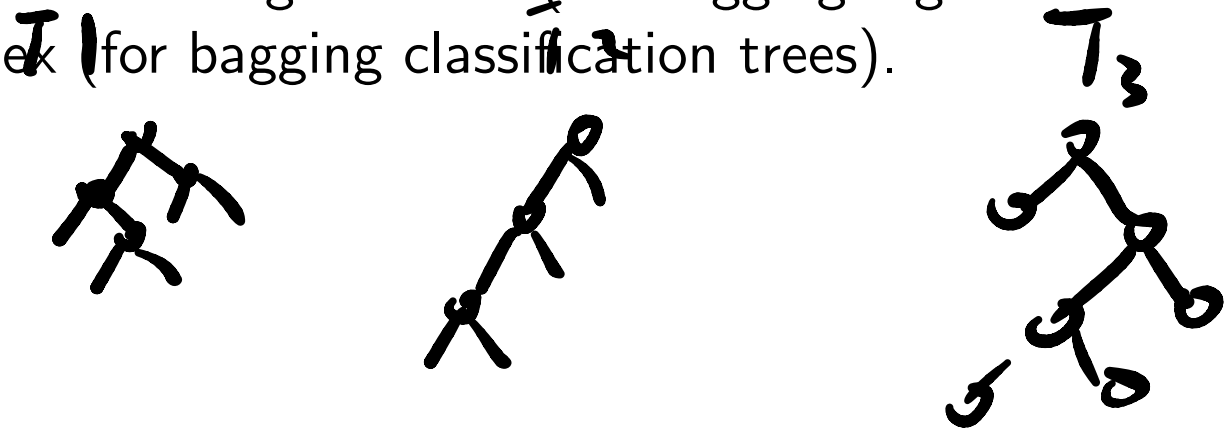
$$i \to B_i \subseteq \{1,\dots,B\}$$

- For each observation $i$, we can pool its predictions across all $B$ bootstrap data sets where $i$ belongs to the OOB observations.
  - For regression, we average
  - For classification, we vote by majority

- We compute the MSE / misclassification error between predictions and the true responses over all observations.

# Variable Importance Measures

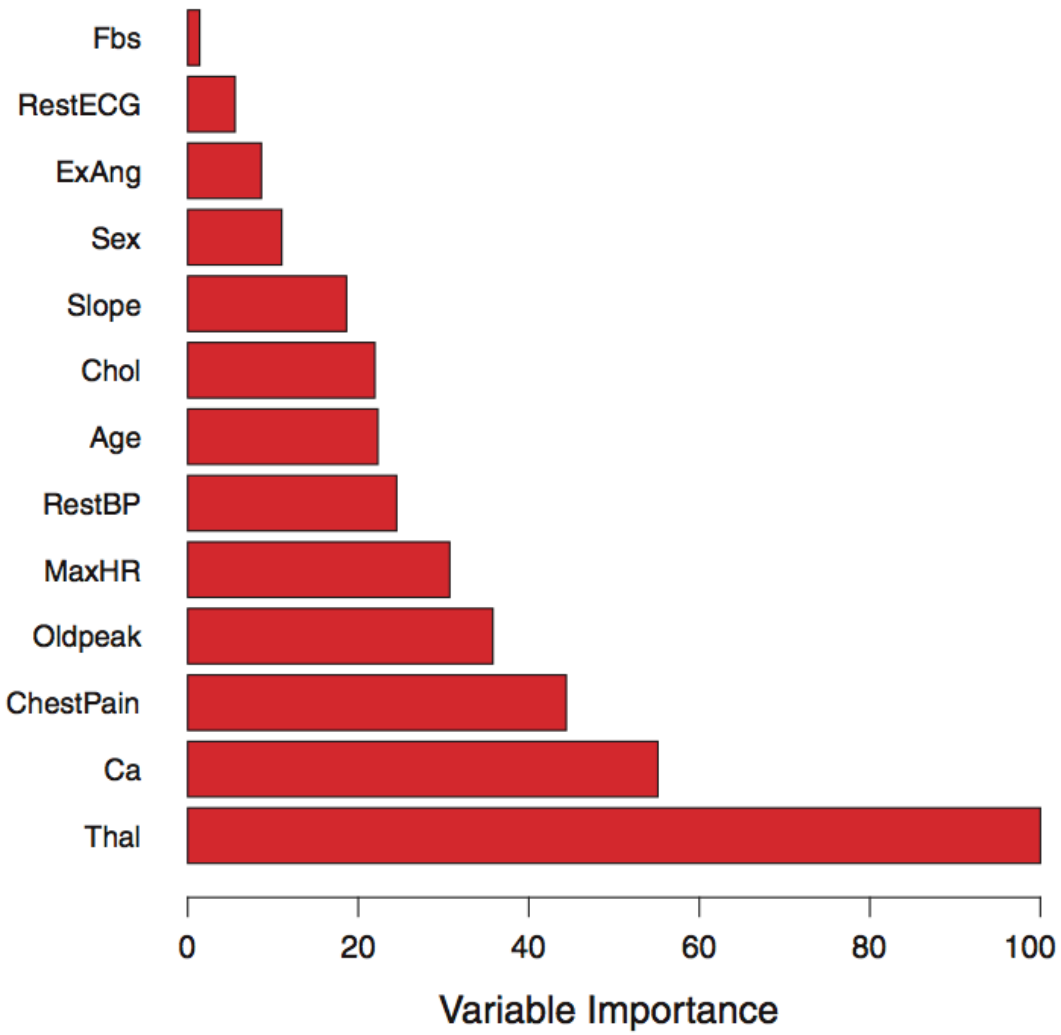
- Bagging typically reduces variance (i.e. improve accuracy) over prediction using a single decision tree.
- However, it can be difficult to interpret the resulting model.
- One can obtain an overall summary of the importance of each predictor using the RSS (for bagging regression trees) or the Gini index (for bagging classification trees).



# Variable Importance Measures

- In the case of bagging regression trees, we can record the total amount that the RSS is decreased due to splits over a given predictor, averaged over all $B$ trees.
  In bagging classification trees, we replace RSS by the Gini index.
- A large value of decreased RSS (or Gini index) indicates an important predictor.
- A graphical representation of variable importance is easy to draw.

# Heart Data




Variable importance is computed using the mean decrease in Gini index, and expressed relative to the maximum.

# Summary on bagging

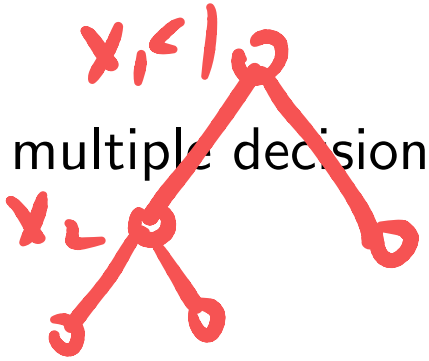
$X_1, X_2, X_3$

$\{X_{i1}, X_{i2}, X_{i3}\}_{1 \le i \le n}$

$X_1 \gg X_2, X_3$

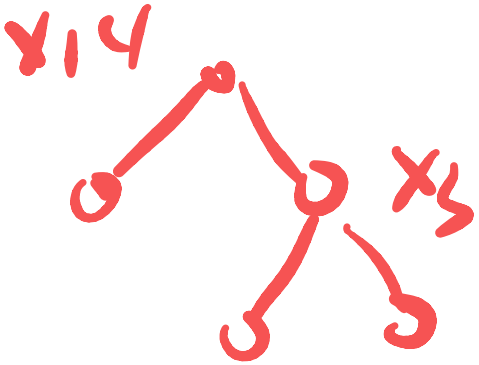
BS-1

$\{X_{i1}^{(1)}, \dots, X_{i3}^{(1)}\}_{1 \le i \le n}$




- Bagging simply averages multiple decision trees
- Bias does not increase
- Variance is reduced
  - **How much is variance reduced?**

BS-2



# Random Forests

- **Bagging is affected by the correlation among decision trees!** [1]
- **Random forests** provide an improvement over bagged trees by decorrelateing the trees. This increases the variance reduction when we average the trees.

[1] Check Prob 3 of the final exam of 2024!

# Random Forests

- As in bagging, we build a number of decision trees on bootstrap training samples.

$\mathrm{Corr}(\hat{f}_1, \hat{f}_2)$

- But for building these decision trees, when each time a split in a tree is considered, a **random selection** of $m < p$ predictors is chosen as split candidates from the full set of $p$ predictors.

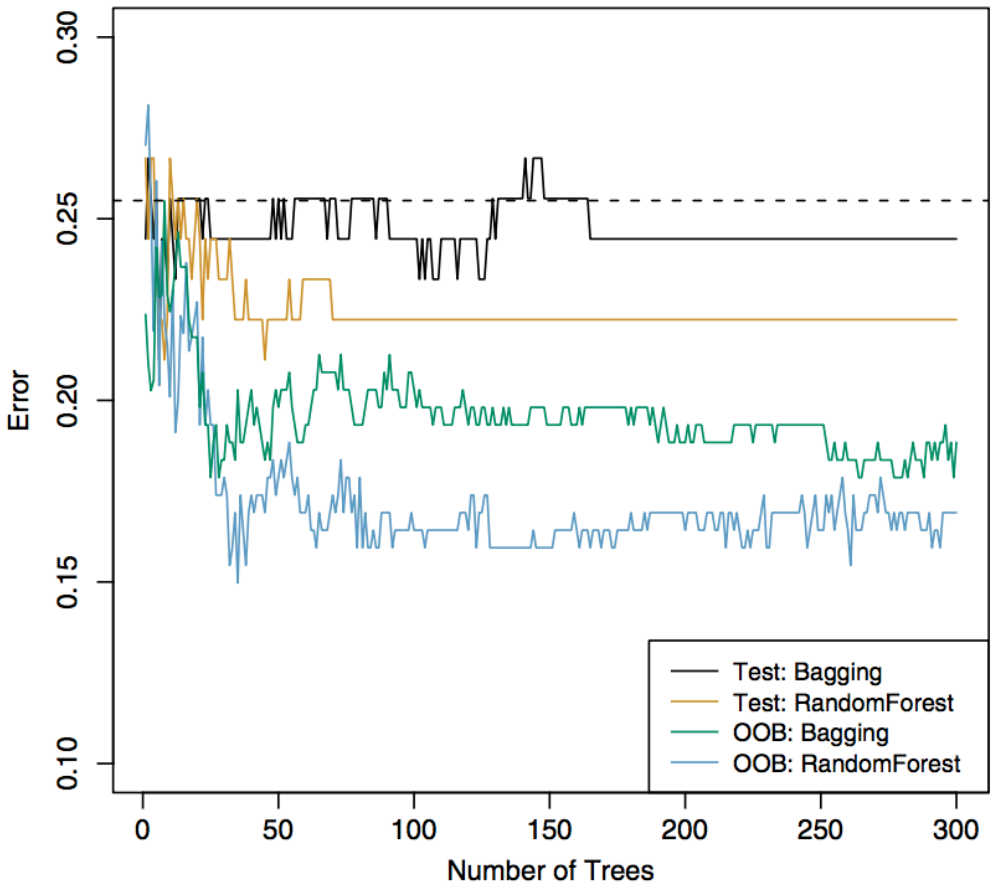
- A fresh selection of $m$ predictors is taken **at each split**, and a common choice of $m$ is $m \approx \sqrt{p}$.

- Random forest with $m = p$ is just bagging!

# Why does Random Forest Reduce Correlations?

- Suppose that there is one very strong predictor in the data set, along with a number of other moderately strong predictors.
- Among all single decision trees, most of them will use this strong predictor in the top split. All of the single trees will look quite similar and could have high correlation.
- Random forests reduce the correlation by randomly selecting a subset of the predictors in each split of buiding each single tree. Indeed, on average many splits will not even consider the strong predictor.

# Heart Data

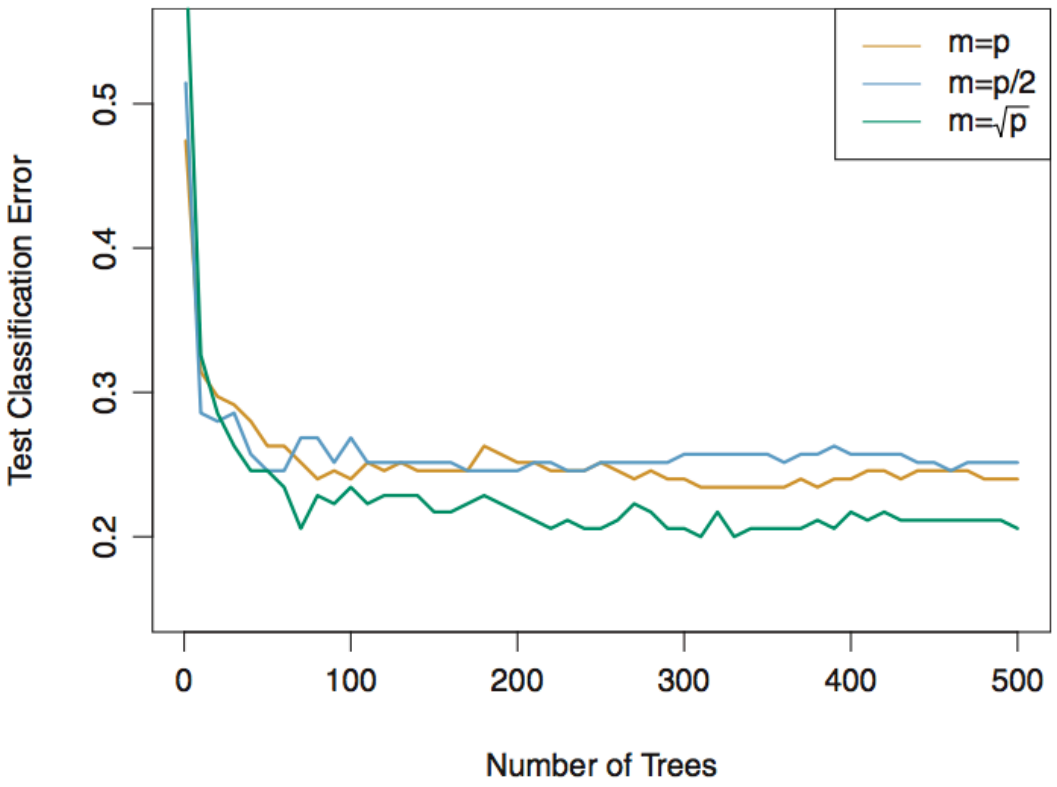



The test error (black and orange) is shown as a function of $B$. Random forests were applied with $m = \sqrt{p}$. The dashed line indicates the test error resulting from a single classification tree. The green and blue traces show the OOB error, which is considerably lower.

# Gene Expression Data

- We applied random forests to a high-dimensional biological data set consisting of expression measurements of 4,718 genes measured on tissue samples from 349 patients.
- There are around 20,000 genes in humans, and individual genes have different levels of activity, or expression, in particular cells, tissues, and biological conditions.
- Each of the patient samples has a qualitative label with 15 different levels: either normal or one of 14 different types of cancer.
- We use random forests to predict cancer type based on the 500 genes that have the largest variance in the training set.
- We randomly divided the observations into a training and a test set, and applied random forests to the training set for three different values of the number of splitting variables $m$.

# Gene Expression Data




The test errors are displayed as functions of the number of trees ($B$).
Random forests ($m < p$) lead to a slight improvement over bagging ($m = p$).
A single classification tree has an error rate of 45.7%.

# Boosting

- Like bagging, boosting is a general approach that can be applied to many statistical learning methods for regression and classification. We focus on the context of decision trees.
- Bagging (and random forest) involves creating multiple copies of the original training data set using bootstrap, fitting a separate decision tree to each copy, and then combining all of the trees in order to create a single predictive model.
- Boosting works both similarly and also differently
  - a Boosting combines different fitted trees
  - b the trees in boosting are grown **sequentially**: each tree is grown using information from previously grown trees
  - b Boosting does not involve bootstrap sampling; instead each tree is fitted on a modified version of the original data set.

# Boosting

**Algorithm 8.2** *Boosting for Regression Trees*

1. Set $\hat{f}(x) = 0$ and $r_i = y_i$ for all $i$ in the training set.
2. For $b = 1, 2, \ldots, B$, repeat:
   (a) Fit a tree $\hat{f}^b$ with $d$ splits ($d+1$ terminal nodes) to the training data $(X, r)$.
   (b) Update $\hat{f}$ by adding in a shrunken version of the new tree:
   $$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x). \tag{8.10}$$
   (c) Update the residuals,
   $$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i). \tag{8.11}$$
3. Output the boosted model,
   $$\hat{f}(x) = \sum_{b=1}^{B} \lambda \hat{f}^b(x). \tag{8.12}$$

$$\begin{matrix}(Y_1, X_1) \\ \vdots \\ (Y_{10}, X_{10})\end{matrix} \rightarrow D^{(1)}$$

$\hat{f} \rightarrow$ decision tree with 2 splits

$$\min_f L(y_i, \hat{y}_i + f(x_i))$$

(1) Obtain $\hat{f}^{(1)}$ using $D^{(1)}$

$$x \rightarrow \hat{f}^{(1)}(x)$$

$$\begin{matrix} x_1 \\ \vdots \\ x_{10} \end{matrix} \rightarrow \begin{matrix} \hat{f}^{(1)}(x_1) \\ \vdots \\ \hat{f}^{(1)}(x_{10}) \end{matrix}$$

$$Y_1^{(2)} := r_1 = y_1 - \eta \underline{\hat{f}^{(1)}(x_1)}$$

$$\vdots$$

$$Y_{10}^{(2)} := r_{10} = y_{10} - \eta \hat{f}^{(1)}(x_{10})$$

$$D^{(2)} = \begin{bmatrix} Y_1^{(2)} & X_1 \\ & \vdots \\ Y_{10}^{(2)} & X_{10} \end{bmatrix}$$

(2) Obtain $\hat{f}^{(2)}$ using $D^{(2)}$

$$\left(\begin{array}{ll} Y_1^{(3)} := Y_1^{(2)} - \eta \hat{f}^{(2)}(x_1) & x_1 \\ \vdots & \vdots \\ Y_{10}^{(3)} := Y_{10}^{(2)} - \eta \hat{f}^{(2)}(x_{10}) & x_{10} \end{array}\right) \to D^{(3)}$$

$\vdots$

Until we obtain $\hat{f}^{(k)} \leftarrow D^{(k)}$

$\forall x$, predict $\eta \sum_{k=1}^{K} \hat{f}^{(k)}(x)$

# Boosting

XG-Boosting

- Unlike fitting a single large decision tree to the data, which might lead to overfitting, the boosting approach instead **learns slowly**.
- Given the current model, we fit a decision tree to the **residuals** from the model. We then add this new decision tree into the fitted function and update the residuals.
- Each of these trees can be rather small, with just a few terminal nodes, determined by the parameter $d$ in the algorithm.
- By fitting small trees to the residuals, we slowly improve $\hat{f}$ in areas where it does not perform well. The step size $\lambda$ further slows down the learning process.

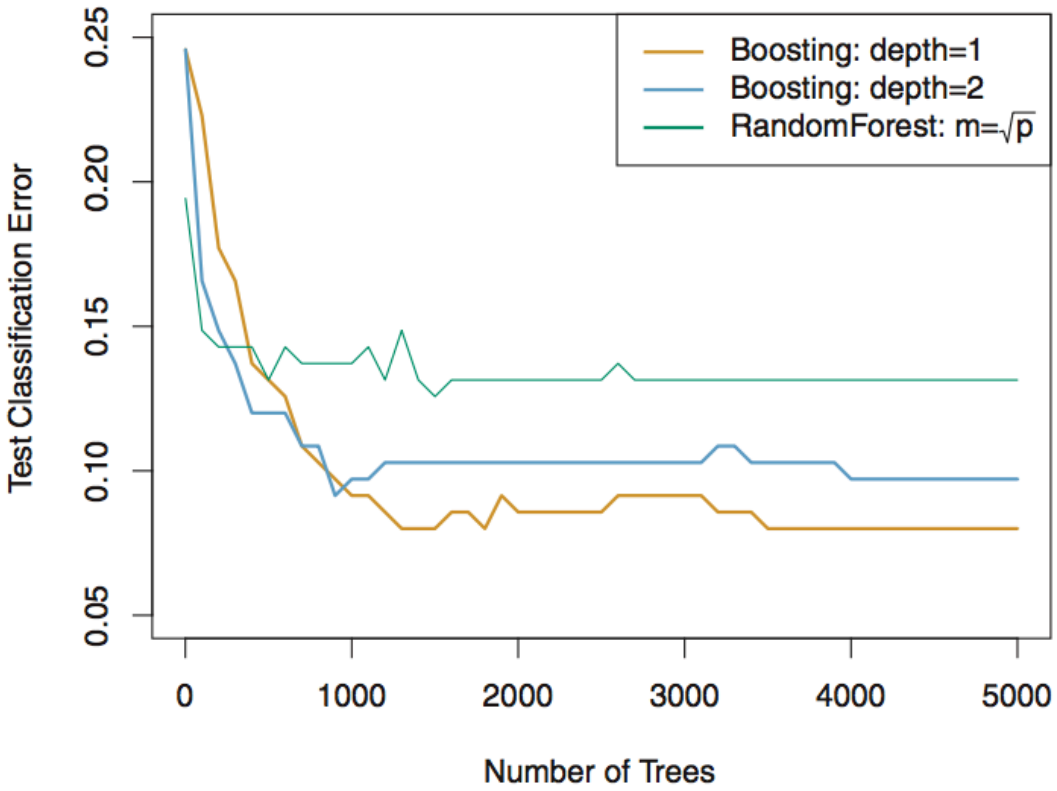
# Tuning parameters for Boosting

- **The number of trees $B$.** Unlike bagging and random forests, boosting can overfit if $B$ is too large, although this overfitting tends to occur slowly if at all. We use cross-validation to select $B$.

- **The shrinkage parameter (step size) $\lambda$**, a small positive number. This controls the rate at which boosting learns. Typical values are 0.01 or 0.001.

- **The number of splits $d$ in each tree**, which controls the complexity of the boosted ensemble. Often $d = 1$ works well, in which case each tree is a **stump**, consisting of a single split and resulting in an additive model.

# Gene Expression Data




For both boosting trees, $\lambda = 0.01$. Depth-1 trees slightly outperform depth-2 trees and random forests. But these differeces are within their standard errors.

# Summary

- Decision trees are simple and interpretable models for regression and classification.
- However their prediction accuracy is often not competitive with other regression / classification methods.
- Bagging, random forests and boosting can improve prediction accuracy of trees. They work by growing many trees on the training data and then combining the predictions of the resulting trees.
- The latter two methods – **random forests** and **boosting** – are among the state-of-the-art methods for supervised learning. However their results can be difficult to interpret.